



Bangladesh University of Engineering and Technology

Course No: EEE 312

Course Name: Digital Signal Processing I Lab

Project Report

Project Name: Keyword Identification for Speech in Matlab

Group No: 02

Group Members:

Md Mostaqul Islam	ID : 1906070
Arpa Roy Dostider	ID : 1906071
Ramisa Tahsin Shreya	ID : 1906072
Md Navid Newaz	ID : 1906073

Course Teachers:

Shahed Ahmed	Barproda Halder
Lecturer	Part-Time Faculty
Dept. of EEE, BUET	Dept. of EEE, BUET

Date of Submission: 08 March 2023

Problem Statement:

The problem introduced to us for the project was keyword identification for speech. We fixed dataset of 6 words, and our program had to identify which keyword was uttered from any person from speech input. The selected words were: “Tennis”, “Golf”, “Soccer”, “Darts”, “Rugby”, “Chess”.

Solution:

First of all, we had to collect data for each keyword. Then, we needed to extract the features so that the program will be able to identify each keyword distinctively. After that, we needed to compare the input with features so that the program can recognize which keyword was uttered.



We wrote a matlab code and circulated that to collect speech data of our classmates. At first we took 8000Hz as sampling frequency but then we found out that speech data should be sampled with 44100Hz. So, we faced a bit challenge there and had to collect data twice.

MFCC is the algorithm we used to extract features from the collected data. We approach this algorithm in two ways: at first, we wrote MFCC code from scratch and then we used built-in MFCC function in matlab. As built-in function gave more accurate results, we stuck with that version.

After taking speech input, we calculated the mfcc of that input and compared that value using dtw with mfcc of the keywords. When the dtw value is less than other keywords, the program identifies the input as that keyword.

Proposed Methodology:

Feature Extraction:

MFCC (Mel Frequency Cepstral Coefficients) is one of the extensively used feature extraction method in speech recognition. MFCC highly resembles the human auditory perception system as it uses logarithmic scale rather than linear scale in frequency analysis.

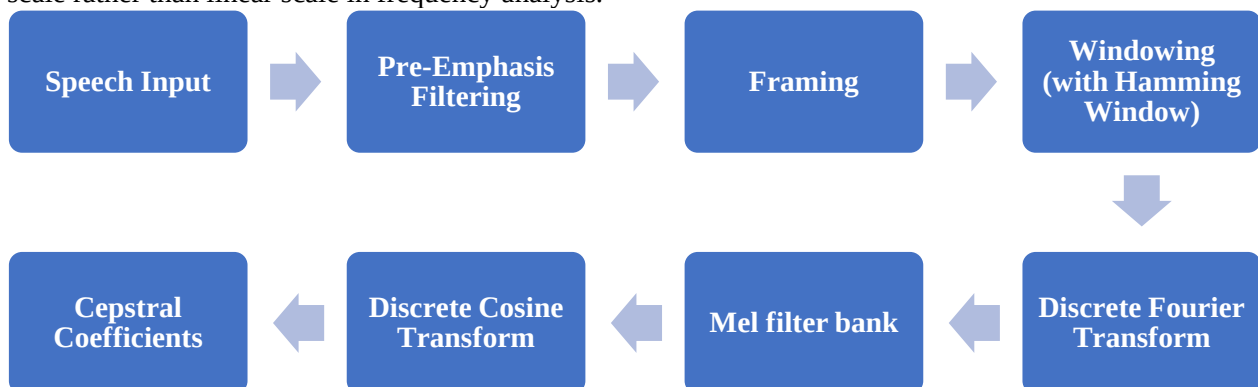


Fig: Flow chart of MFCC Feature Extraction

Working: Pre-emphasis filter first filters out some of the effects of vocal tract parameter. Speech is then separated in short periods of time (typically 20-30 ms) with 10 ms overlapping period for attaining stable characteristics. Hamming window is applied on each frame to reduce the discontinuities. Each windowed frame is then converted into magnitude spectrum using N-point DFT (Discrete Fourier Transform). The linear frequency scale is then transformed into mel scale using the equation: $Mel(f) = 2595 \log_{10}(1 + \frac{f}{700})$. A set of triangular band pass filters (mel filter bank) is formed and the Fourier Transformed signal is passed through this in order to smooth the magnitude spectrum. At last, Discrete Cosine Transform (DCT) is used to calculate the cepstral coefficients. Total 14 coefficients are extracted in this process. We eliminated the zero order coefficient, that is, average power and log energy as they are less significant in keyword recognition. For a given audio sample, we found a 298×12 matrix at the end of feature extraction where each row corresponds to each frame and each column corresponds to a feature.

Manual Code for MFCC:

```
[x,fs]=audioread('F:\Level-3 Term-1\EEE312\Final Project\Chess\Chess1.wav');
%Amplification
if( max(abs(x))<=1 )
    x = x * 2^15;
end
%pre-emphasis filter
a=[1];
alpha=-0.97;
b=[1 alpha];
y=filter(b,a,x);
%framing
frame_duration=0.025;
frame_len = frame_duration*fs;
framestep=0.01;
framestep_len=framestep*fs;
frames=[];
```

```
j=1;
while (((j-1)*framestep_len)+frame_len)<=3*fs
    frame=y((j-1)*framestep_len + 1: ((j-1)*framestep_len)+frame_len,1)';
    frames=[frames;frame];
    j=j+1;
end
frame_len=length(frame');
%windowing
num_frames=size(frames,1);
h=hamming(frame_len);
for i=1:num_frames
    fr=frames(i,:);
    window(i,:)=fr.*h';
end
%dft
nfft = 2^nextpow2( frame_len );
```

```
for i=1:num_frames
    ft(i,:)=abs(fft((window(i,:)),nfft));
end
ft=ft';
M=20;
noCoeff=13;
N=noCoeff+1;
K=(nfft/2)+1;
% mel frequency conversion
hz2mel = @( hz ) ( 2595*log(1+hz/700) ); %
Hertz to mel
mel2hz = @( mel ) ( 700*exp(mel/2595)-700 ); %
mel to Hertz
f_min = 0;
f_max = 0.5*fs;
f = linspace( f_min, f_max, K );
fw = hz2mel( f );
c = mel2hz(
    hz2mel(f_min)+[0:M+1]*((hz2mel(f_max)-
    hz2mel(f_min))/(M+1)) );
cw = hz2mel( c );
```

```
%mel filter bank
H = zeros( M, K );
for m = 1:M
    k = f>=c(m)&f<=c(m+1); % up-slope
    H(m,k) = (f(k)-c(m))/(c(m+1)-c(m));
    k = f>=c(m+1)&f<=c(m+2); % down-slope
    H(m,k) = (c(m+2)-f(k))/(c(m+2)-c(m+1));
end
plot(f,H);
xlabel('frequency');
title('20 mel filter banks');
%discrete cosine transform
w = H * ft(1:K,:);
DCT = sqrt(2.0/M) * cos( repmat([0:N-1].',1,M).*
    repmat(pi*[1:M]-0.5)/M,N,1) );
CC = DCT * log( w );
% Cepstral lifter to attenuate the higher
coefficients
L=22;
lifter = 1+0.5*L*sin(pi*[0:N-1]/L) ;
mf = diag( lifter ) * CC;
mf=mf';
```

Matching Process:

DTW (Dynamic Time Warping) has been used in this project to find similarities between two time sequences. These may vary in speed (as different speakers have different speaking speed). Considering any two speech patterns, we can get rid of their timing differences by warping the time axis of one so that the maximal coincidence is attained with the other. The main idea of DTW is to compute the distance from the matching of similar elements between two time series. DTW compares the amplitude of the first signal at time T with the amplitude of the second signal at time T+1 and T-1 or T+2 and T-2. This makes sure it does not give a low similarity score for signals with similar shapes and different phases. Thus, it gives better comparison than Euclidean distance or any other distance.

Working: Let, for two time series A and B,

Create an empty matrix M and label with the amplitudes of two series.

Starting from the bottom left corner, fill the matrix with the formula: $M(i, j) = |A(i) - B(j)| + \min(M(i-1, j-1), M(i, j-1), M(i-1, j))$

Find the warping path starting from the right top corner and traversing to left bottom corner, which is identified based on the neighbor with a minimum value.

Calculate normalized distance using the formula $D = \frac{\sum d_i}{k}$; d=warping path series, k= Length of d

Results and Explanation:

So far, we have tried so many algorithms for feature extraction and matching. All the efforts behind these algorithms and the corresponding results with explanation are given below:

Feature Extraction Algorithms and Corresponding Results:

1. MFCC:

- MFCC by user defined function-> Easy to understand the insight of MFCC but execution time is poor. Takes too long time to read and extract 172 files and also to train them.
- MFCC by built in function-> Difficult to visualize the processing behind but execution time is very quick. That's why built in MFCC is used.
- MFCC excluding the log energy and PSD property: Normal MFCC returns 14 features for each of the window. But the efficiency with 14 properties (with any matching algorithms) is not so good (around 60%). That's why 2 properties log energy and PSD were excluded and the remaining 12 properties were kept and accuracy was so good (around 80%).

2. Spectral Roll-off:
 - This algorithm for feature extraction for word matching did not give satisfactory result.
3. Spectral Centroid:
 - This algorithm for feature extraction for word matching did not give satisfactory result.
4. Zero-crossing Rate:
 - This algorithm also for feature extraction for word matching did not give satisfactory result.
5. Pitch:
 - This algorithm extracts speaker dependent features rather than word dependent features. That's why we could not get more than 50% accuracy using this feature to detect words.
6. Window length change:
 - By changing the length of hamming window used in MFCC function, the word detection features can be better extracted. But using ML (Classification Learner App) to classify and distinguish them did not improve the accuracy so much. This will be discussed further in the matching algorithm part.

Matching Algorithms and Corresponding Results:

1. Classification Learner App (Machine Learning):
 - From Matlab built in Classification learner app, there are 29 models for classify different features of same words and vice versa. But, unfortunately none of the model gave us satisfactory accuracy to distinguish six different words. For example, changing window length and classification results are shown below for some of the models:

Name of the Model Used	Accuracy with Hamming window length of 2048 samples	Accuracy with Hamming window length of 1024 samples
Coarse Tree	23.4%	26.7%
Linear Discriminant	40.6%	39.6%
Gaussian Naïve Bayes	28.8%	30.2%
Cubic SVM	41.5%	39.3%
Fine KNN	37.8%	38.7%
RUSBoosted Trees	25.2%	28.9%
Narrow Neural Network	41.1%	39.2%

2. Mahalanobis Distance:
 - Mahalanobis distance is able to calculate similarities between two speech signals even if any relation exists between them. But the problem is Mahal function returns a matrix in stead of a single value. That's why an average has to be taken to match with other mean mahalanobis distances. That's why the accuracy for feature matching drops drastically but this accuracy is better than Euclidean distance.
3. DTW (Dynamic Time Warping):
 - The choice of DTW for best matching algorithm without DL is probably DTW. Because,
 - a. 'dtw' function returns a single value. So, easy to compare.
 - b. DTW can find similarities between 2 signals even if one is zero padded.
 - c. DTW can find similarities between 2 signals even if one is amplified.
 - d. DTW can find similarities between 2 signals even if one is added a DC offset.
 - e. DTW can find similarities between 2 signals even if one is attenuated.

- f. DTW can find similarities between 2 signals even if the signals are multidimensional matrices.
 - That's why the accuracy by DTW matching was almost 80-90% and it was selected as the final matching algorithm for the final presentation file.
4. Deep Learning:
- We have also tried some deep learning algorithms parallelly with others by taking help from mathworks website and somewhat succeeded. But this idea was scraped according to the advice of our respected Lab Teacher although the accuracy could be more than 95%.

Discussion:

Despite all the struggles and mismanagements, we finally was able to execute our program with expected accuracy following MFCC as feature extracting process and DTW as matching algorithm just 14 hours before the presentation which was one of the most pleasuring memories of our lives. So, we are really very thankful to our respected lab teachers who gave us such a challenging project to explore the real BUETians inside each of us and enable us to gather a vast of amount of knowledge on digital signal processing/DSP.

List of References:

- [1] MFCC Features_Springer Link, Accessed: March 6, 2023. [Online]. Available: <https://link.springer.com/content/pdf/bbm:978-3-319-49220-9/1.pdf>
- [2] S A. Valaki, H B. Jethva." A Survey on Feature Extraction and Classification Techniques for Speech Recognition" ijariie.com, Accessed: March 7, 2023. http://ijariie.com/AdminUploadPdf/A_Survey_on_Feature_Extraction_and_Classification_Techniques_for_Speech_Recognition_ijariie3432.pdf
- [3] A H. Poorjam. "Why we take only 12-13 MFCC coefficients in feature extraction?" researchgate.net, Accessed: March 5, 2023. https://www.researchgate.net/post/Why_we_take_only_12-13_MFCC_coefficients_in_feature_extraction
- [4] N S Chauhan. "Dynamic Time Warping (DTW) Algorithm in Time Series" theaidream.com, Accessed: March 8, 2023. <https://www.theaidream.com/post/dynamic-time-warping-dtw-algorithm-in-time-series>
- [5] Train Speech Command Recognition Model Using Deep Learning: March 8, 2023. [Online]. Available: <https://www.mathworks.com/help/deeplearning/ug/deep-learning-speech-recognition.html>